InterFormer: Effective Heterogeneous Interaction Learning for Click-Through Rate Prediction (2025)

Challenges

- **Insufficient inter-mode interaction.** Prior heterogeneous CTR models often have *unidirectional* information flow between non-sequential (profile/content) features and sequential (behavior) features, limiting "mutual benefits" across modes.
- **Aggressive information aggregation.** Many approaches summarize sequences early, which can cause *excessive information loss* before sequence context can influence non-sequence interaction learning.
- **Noisy/multi-source sequences.** Real systems may have multiple behavior sequences (click, conversion, etc.) with noise and varying lengths; naïve fusion can degrade both effectiveness and efficiency.

Key Initiatives

- **Interleaving (bidirectional) heterogeneous interaction learning.** InterFormer is designed so that sequence and non-sequence representations are updated in a mutually conditioning way across layers.
- **Retain full-token representations per mode; summarize only for exchange.** Instead of early pooling, InterFormer keeps the dimensionalities of each mode through stacked layers and uses a *separate Cross Arch* to selectively summarize information for cross-mode exchange.
- **Selective summarization via gating + multiple sequence summaries.** Non-sequence tokens are summarized with a gated selection; sequences are summarized with a combination of CLS tokens, PMA tokens, and recent tokens, then gated.

Methodology

- **Inputs / preprocessing.** Dense and sparse non-sequence features are embedded into a unified d-dim space. Each sequence event is embedded; multiple sequences can be unified/denoised using MaskNet-style self-masking.
- **Interaction Arch (behavior-aware non-seq interactions).** At each layer, a backbone interaction module (e.g., DOT/FM-like, DCNv2, DHEN) models interactions over the concatenation of non-seq tokens and a *sequence summary* $S_{\text{sum}}^{(\ell)}$, followed by an MLP shape-matching transform to preserve token-level mappings for stacking.
- **Sequence Arch (context-aware sequence modeling).** Sequence tokens are updated using (i) *Personalized FFN (PFFN)* conditioned on a non-seq summary $X_{\text{sum}}^{(\ell)}$, and (ii) *Multi-Head Attention (MHA)* to capture long-range dependencies. The first layer prepends $X_{\text{sum}}^{(1)}$ as a CLS token so attention can aggregate sequence info using non-seq context as query; rotary position embeddings are applied.
- **Cross Arch (selection + exchange).** To make exchange effective and efficient, the model computes low-dimensional summaries before exchanging:
  - Non-seq summary: $X_{\text{sum}}^{(\ell)} = \text{Gating}(\text{MLP}(X^{(\ell)}))$.
  - Seq summary: $S_{\text{sum}}^{(\ell)} = \text{Gating}(S_{\text{CLS}}^{(\ell)} \parallel S_{\text{PMA}}^{(\ell)} \parallel S_{\text{recent}}^{(\ell)})$.
- **Design intent.** The authors emphasize separating "information summarization" from the two main arches to avoid aggressive aggregation while still enabling cross-mode interaction.

Experiments

- **Datasets.** Three public benchmarks: Amazon-Electronics (3.0M samples), TaobaoAd (25.0M), KuaiVideo (13.7M), plus an internal Meta dataset.
- **Baselines.** Non-seq: FM, xDeepFM, AutoInt+, DCNv2, FmFM, DOT, DHEN, Wukong. Seq: DIN, DIEN, BST, DMIN, DMR, TransAct.
- **Metrics.** AUC / gAUC, LogLoss; industrial: Normalized Entropy (NE) and system metrics (QPS, MFU).
- **Headline results (paper-quoted).** "InterFormer achieves up to 0.14% AUC improvement on benchmark datasets and 0.15% Normalized Entropy (NE) gain on industrial dataset."
- **Industrial scale (paper-quoted).** The internal dataset is described as "containing 70B samples in total, hundreds of non-sequence features and 10 sequences of length 200 to 1,000." A 3-layer InterFormer reports "a 0.15% NE gain" and "24% Queries Per Second (QPS) gain" vs internal SOTA with similar FLOPs.
- **Ablations/analysis.** The paper tests (i) interleaving vs uni-directional/disabled flows, supporting the bidirectional benefit; (ii) selective aggregation vs average pooling/MLP/MHA early summarization; (iii) scaling with more sequence features and more layers.

References (selected)

- Transformer / attention for sequence modeling: Vaswani et al. (Transformer); DIN, DIEN; BST.
- Feature interaction baselines: FM; DCNv2; xDeepFM; AutoInt; DHEN; Wukong.
- Related industrial sequential CTR systems: TransAct; LiRank.

arXiv: 2411.09852

Table 1: Symbols and notations.

Symbol	Definition
$\mathcal{U}, \mathcal{I}$	user set and item set
$\mathbf{x}_j^{(l)}$	j-th non-sequence feature of layer l
$\mathbf{s}_t^{(l)}$	sequence feature of layer l at time step t
m, n, k	number of dense, sparse and sequence features
T, d	sequence length and embedding dimension
$\odot$	Hadamard product
$\langle \cdot, \cdot \rangle$	inner product
$[\cdot \| \cdot]$	horizontal concatenation of vectors

item i^u_{T+1}, denoted as $y^u_{i_{T+1}}$, given the historical interaction sequence S^u. Formally, we seek to learn a function $f : \mathcal{U} \times \mathcal{I} \times \mathcal{S} \rightarrow [0, 1]$, where $\mathcal{S}$ is the set of all possible sequences, such that:

$$P(y^u_{i_{T+1}} = 1 | u, i_{T+1}, S^u; \theta) = f(u, i_{T+1}, S^u; \theta). \tag{1}$$

To optimize the model, the cross-entropy loss, which measures the difference between predicted and groundtruth click probabilities, is commonly employed as the objective function. Since the formulation in Eq. (1) incorporates temporal dynamics in user behaviors, it potentially leads to better CTR predictions than non-sequential methods that ignore the sequential nature of user interactions.

3.2 Feature Interaction

Interaction learning, which aims to capture the complex relationships between different features, is the key towards the success of CTR prediction. We briefly introduce three prominent feature interaction modules: inner product, DCNv2 [47] and DHEN [79].

Inner Product-based Interaction. Given input feature vector $\mathbf{x} \in \mathbb{R}^d$, the inner product-based interaction, exemplified by Factorization Machines (FM) [35], learns a latent vector $\mathbf{v}_j \in \mathbb{R}^r$ for each feature j, whose inner product $\langle \mathbf{v}_j, \mathbf{v}_k \rangle$ describes the interaction strength between features $\mathbf{x}(j)$ and $\mathbf{x}(k)$. Hence, the second-order feature interactions can be modeled as:

$$f_{FM}(\mathbf{x}) = \sum_{j=1}^{d} \sum_{k=j+1}^{d} \langle \mathbf{v}_j, \mathbf{v}_k \rangle \mathbf{x}(j)\mathbf{x}(k) + \sum_{j=1}^{d} w_j \mathbf{x}(j) + w_0.$$

Deep & Cross Network (DCNv2) [47]. The DCNv2 model combines a cross network for explicit feature interactions with a deep neural network for implicit feature interactions. Given input feature vector $\mathbf{x}^{(0)} \in \mathbb{R}^d$, the l-th layer of the cross network models the second order interaction as follows:

$$\mathbf{x}^{(l+1)} = \mathbf{x}^{(0)} \odot \left(\mathbf{w}^{(l)} \mathbf{x}^{(l)} + \mathbf{b}^{(l)} \right) + \mathbf{x}^{(l)}.$$

DCNv2 further combines the stacked cross layers for explicit interactions with a deep layer for implicit interactions, enabling an output enriched with comprehensive information.

Deep Hierarchical Ensemble Network (DHEN) [79]. DHEN learns a hierarchy of interactions based on a layered structure where each layer is an ensemble of multiple heterogeneous interaction modules. Given the concatenation of m input features $\mathbf{X}^{(l)} \in \mathbb{R}^{d \times m}$, the output of the l-th DHEN layer is computed as:

$$\mathbf{X}^{(l+1)} = \text{Norm}\left(\text{Ensemble}_{i=1}^{k} \text{Interaction}_i(\mathbf{X}^{(l)}) + \text{ShortCut}(\mathbf{X}^{(l)}) \right),$$

where $\text{Norm}(\cdot)$ is a normalization function, Interaction_i represents different interaction modules that are further assembled by $\text{Ensemble}_{i=1}^{k}(\cdot)$ such as summation and concatenation, and $\text{ShortCut}(\cdot)$ is an MLP serving as the residual connection for deep layer stacking.

3.3 Attention Mechanism

Attention mechanisms [1] have become an integral part of sequence modeling, serving as the core component behind various model designs [7, 43]. We briefly review attention mechanisms, including Multi-Head Attention and Pooling by Multihead Attention (PMA).

Multi-Head Attention (MHA) [43] allows sequence modeling without regard to distance between tokens [43]. Given an input sequence $\mathbf{S} = [\mathbf{s}_1, \ldots, \mathbf{s}_T]$, where $\mathbf{s}_t \in \mathbb{R}^d$ is the embedding vector at timestamp t, the self-attention operation is defined as:

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\mathbf{Q}\mathbf{K}^\intercal / \sqrt{d_k} \right) \mathbf{V}, \tag{2}$$

where $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{n \times d_k}$ are query, key and value representations computed by linear projections $\mathbf{Q} = \mathbf{S}^\intercal \mathbf{W}^Q$, $\mathbf{K} = \mathbf{S}^\intercal \mathbf{W}^K$, $\mathbf{V} = \mathbf{S}^\intercal \mathbf{W}^V$.

To enable joint attention to information from different embedding subspaces, MHA aggregates h parallel attention as follows:

$$\text{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = [\mathbf{head}_1 \| \ldots \| \mathbf{head}_h] \, \mathbf{W}^O, \tag{3}$$

where heads are computed by self-attention $\mathbf{head}_i = \text{Attn}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$ and aggregated via an output projector $\mathbf{W}^O \in \mathbb{R}^{hd_k \times d}$.

Pooling by Multi-Head Attention (PMA) [17]. Rather than obtaining query, key and values from the same input sequence $\mathbf{S}$, PMA utilizes a *learnable* query $\mathbf{Q}_{\text{PMA}} \in \mathbb{R}^{k \times d_k}$ to summarize the sequence from k different aspects, which can be defined as follows

$$\text{PMA}(\mathbf{Q}_{\text{PMA}}, \mathbf{S}) = \text{MHA}(\mathbf{Q}_{\text{PMA}}, \mathbf{K}, \mathbf{V}). \tag{4}$$

Intuitively, each column in $\mathbf{Q}_{\text{PMA}}$ is a seed vector summarizing $\mathbf{S}$ as a d-dimensional vector, and the output of PMA is the concatenation of k summarization depicting sequence from different aspects.

4 Methodology

In this section, we present our proposed InterFormer. We first introduce the preprocessing module in Section 4.1, followed by three major modules, including Interaction Arch, Sequence Arch and Cross Arch. To learn behavior-aware non-sequence embeddings, the Interaction Arch models feature interactions (Section 4.2). To learn context-aware sequence embeddings, the Sequence Arch is proposed to optimize sequence (Section 4.3). The Cross Arch connects Interaction and Sequence Arches, enabling effective information summarization and exchange between different modes (Section 4.4). An overview of InterFormer is shown in Figure 1.

4.1 Feature Preprocessing

Non-Sequence Feature Preproessing. Two types of non-sequence features are considered, including dense features like user age and item price, and sparse features like user id and item category. To unify the heterogeneity in non-sequence features, different features are transformed into embeddings of the same dimensionality.

Specifically, raw dense features $x^{(0)}_{\text{dense}_i}$ are first concatenated to form a dense vector $\mathbf{x}^{(0)}_{\text{dense}} = \left[x^{(0)}_{\text{dense}_1}, \ldots, x^{(0)}_{\text{dense}_m} \right]^\intercal$, which is further transformed into a d-dimensional dense embedding vector

Figure 1: An overview of the InterFormer model architecture (from the paper).